

МИНИСТЕРСТВО ТРАНСПОРТА РОССИЙСКОЙ ФЕДЕРАЦИИ

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ УНИВЕРСИТЕТ ТРАНСПОРТА»
(РУТ(МИИТ))**

Институт управления и цифровых технологий

Кафедра «Цифровые технологии управления транспортными процессами»

ЛАБОРАТОРНАЯ РАБОТА №6

По дисциплине «Базы данных и экспертные системы»

Вариант №6

Выполнил:
студент группы УПМ-311
Воробченко А.А

Проверил:
доцент Козьяков П.О.

Москва

2025 г.

Содержание

1. Цель работы.....	3
2. Текст задания.....	3
3. Практическая часть.....	4
4. Вывод.....	27

1. Цель работы

Приобрести навыки реализации и анализа динамических структур данных — сбалансированных деревьев поиска, освоить алгоритмы построения, включения, удаления и перебалансировки узлов в AVL- и B-деревьях, а также научиться выбирать и применять эффективные структуры данных для организации быстрого доступа к информации в задачах хранения и поиска

2. Текст задания

1. Построить AVL-дерево, включая элементы в указанной последовательности (построчно).
2. Из построенного AVL-дерева удалить элементы, обозначенные в исходном списке буквами, в прямой последовательности, а затем и корень.
3. Построить B-дерево (порядка 2 или 3), включая элементы в указанной последовательности по столбцам.
4. Из построенного B-дерева удалить элементы, обозначенные в исходном списке буквами, в обратной последовательности, а затем в корневой вершине удалить крайний правый элемент.

100	80	70	N	85	82	30	L	50	73
55	113	M	75	56	68	125	K	42	61
Z	65	38	29	31	9	92	90	98	170

Для варианта №6:

N — номер студента по списку группы = 6,

$L = 221 - N = 215$,

$M = \frac{L}{2} - N = 102$,

$K = 3 + \text{Ц}\left(10 * \frac{N}{3}\right) = 23$,

$Z = N + n \text{ (имя)} = 12$.

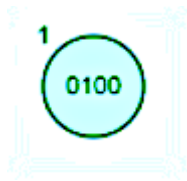
100	80	70	6	85	82	30	215	50	73
55	113	102	75	56	68	125	23	42	61
12	65	38	29	31	9	92	90	98	170

3. Практическая часть

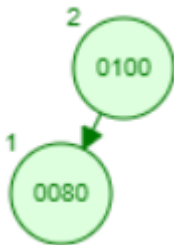
1. Построить AVL-дерево, включая элементы в указанной последовательности (построчно).

Постепенный процесс построения дерева (добавлялось по три элемента):

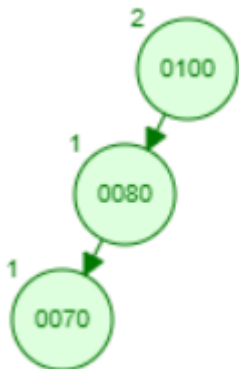
AVL – это дерево поиска. Оно строится по следующему образцу: все элементы слева < узел < все элементы справа. Поэтому все числа меньше 70 оказываются в левом поддереве корня, а все числа больше 70 – в правом. И так рекурсивно в каждом узле. Самое главное свойство в AVL-дереве – сохранение баланса. То есть, для каждой его вершины высота её двух поддеревьев различается не более чем на 1. Корнем становится та вершина, которая оказалась самой устойчивой при балансировке. В моём случае это 70.



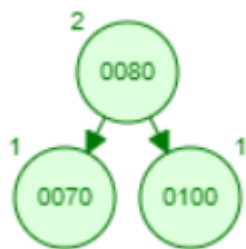
Вставим элемент 80 в левое поддерево, так как $80 < 100$:



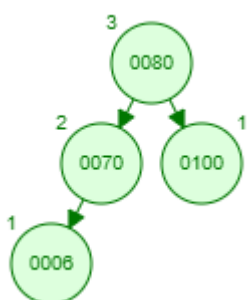
Вставим элемент 70 в левое поддерево, так как $70 < 80$:



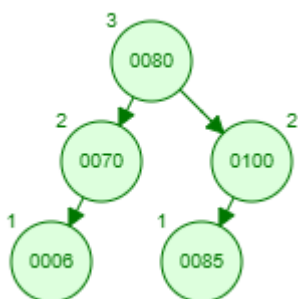
Высота двух поддеревьев отличается больше чем на 1, выполним правый поворот:



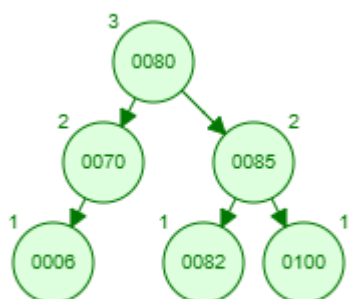
Вставим элемент 6, так как $6 < 70 < 80$:



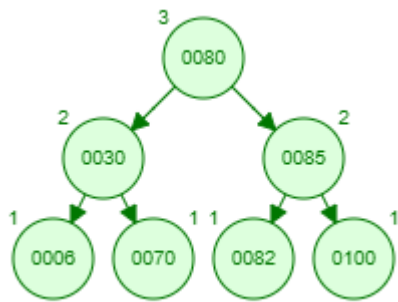
Вставим 85, $85 > 80$, но $85 < 100$:



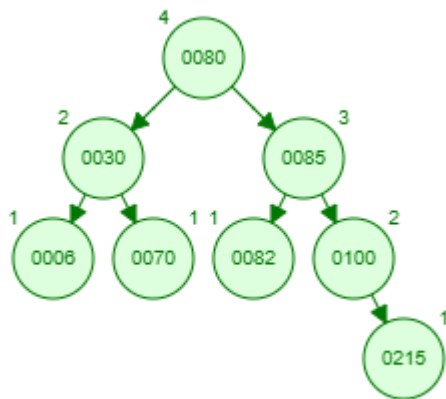
Вставим 82:



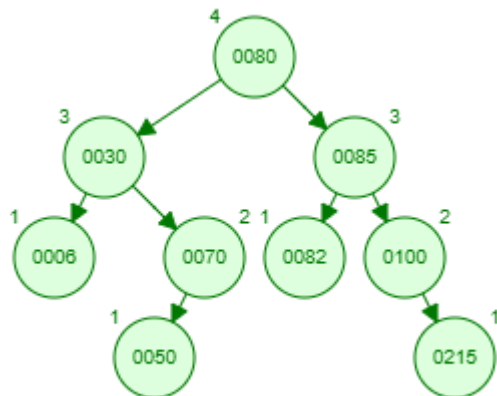
Вставим 30:



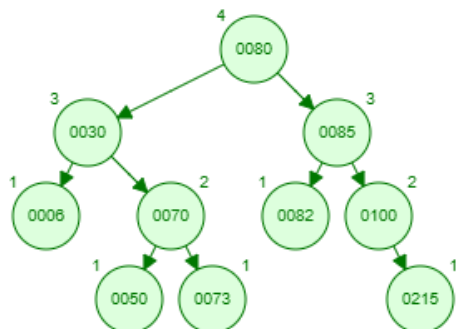
Вставим 215:



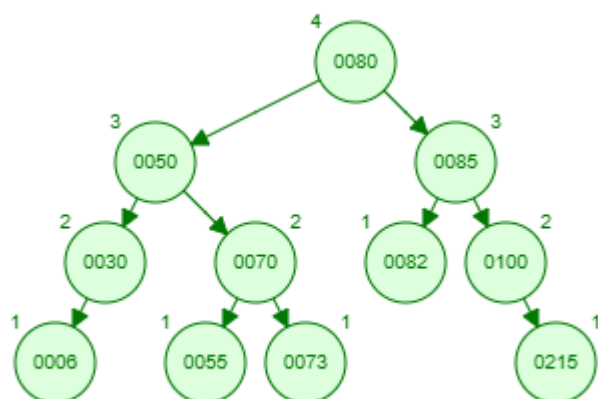
Вставим 50:



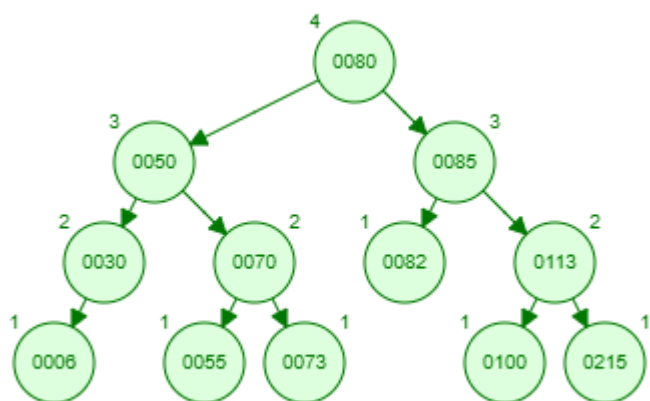
Вставим элемент 73:



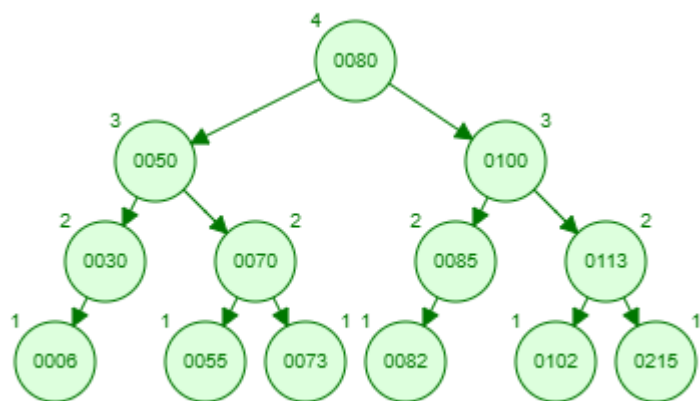
Вставим элемент 55:



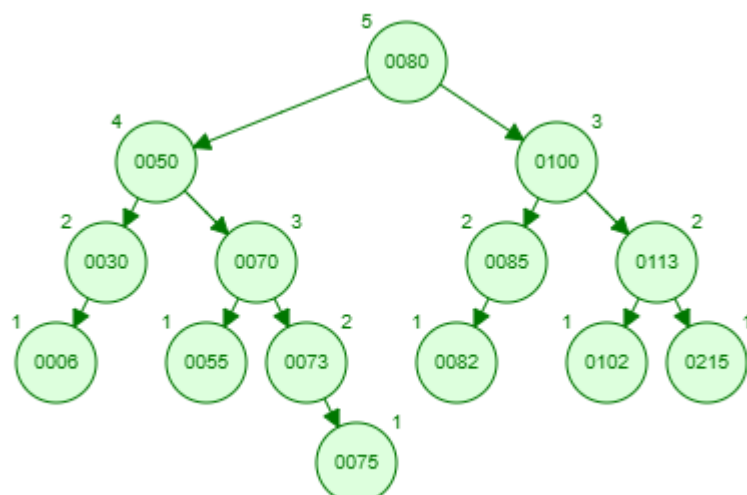
Вставим элемент 113:



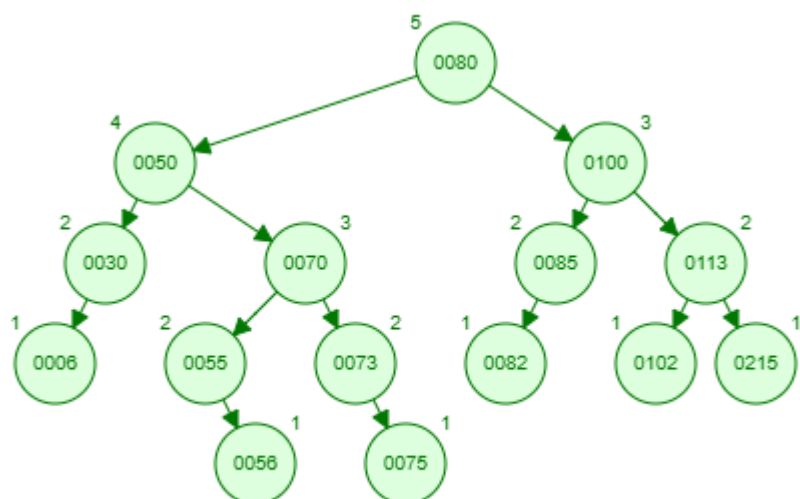
Вставим элемент 102:



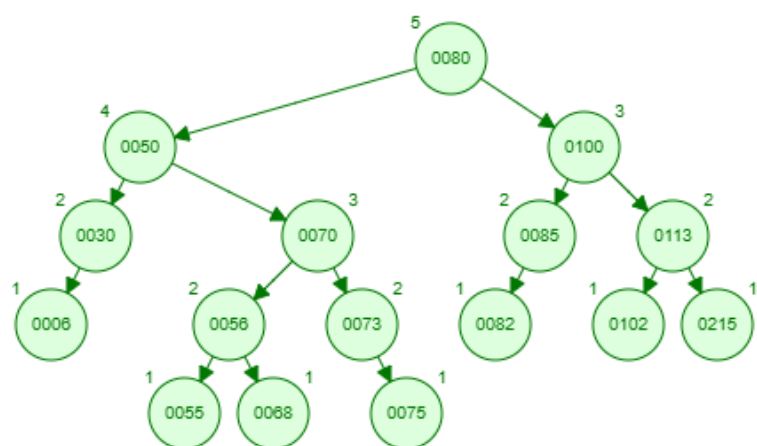
Вставим элемент 75:



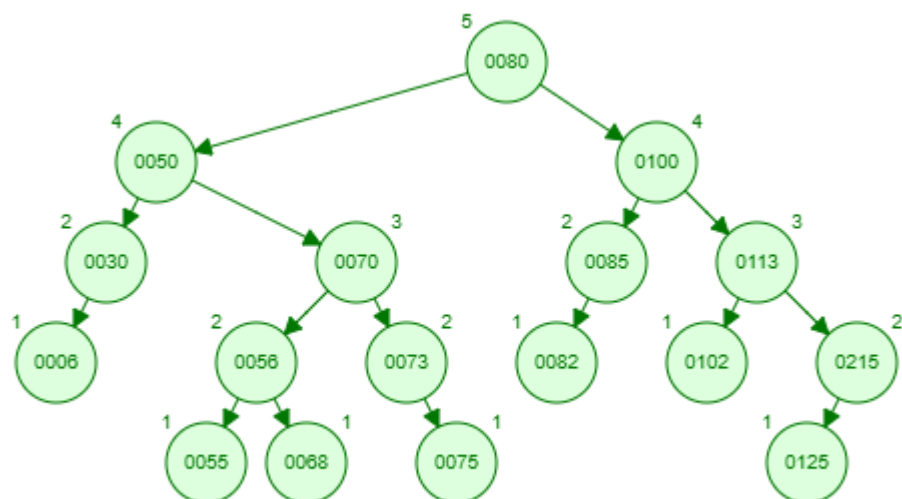
Вставим элемент 56:



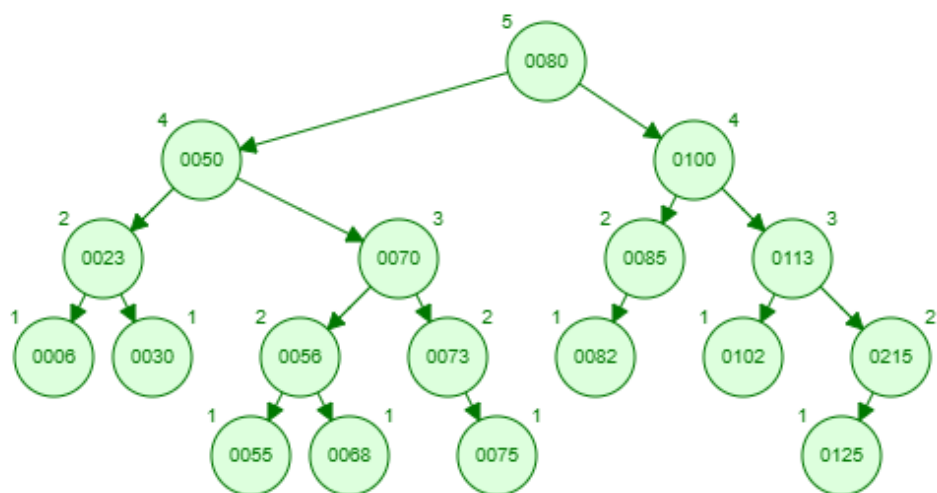
Вставим элемент 68:



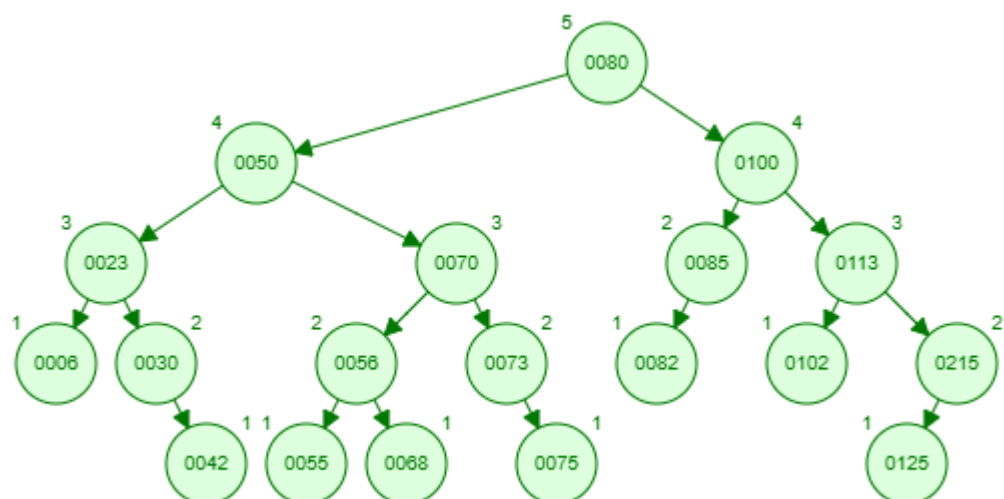
Вставим элемент 125:



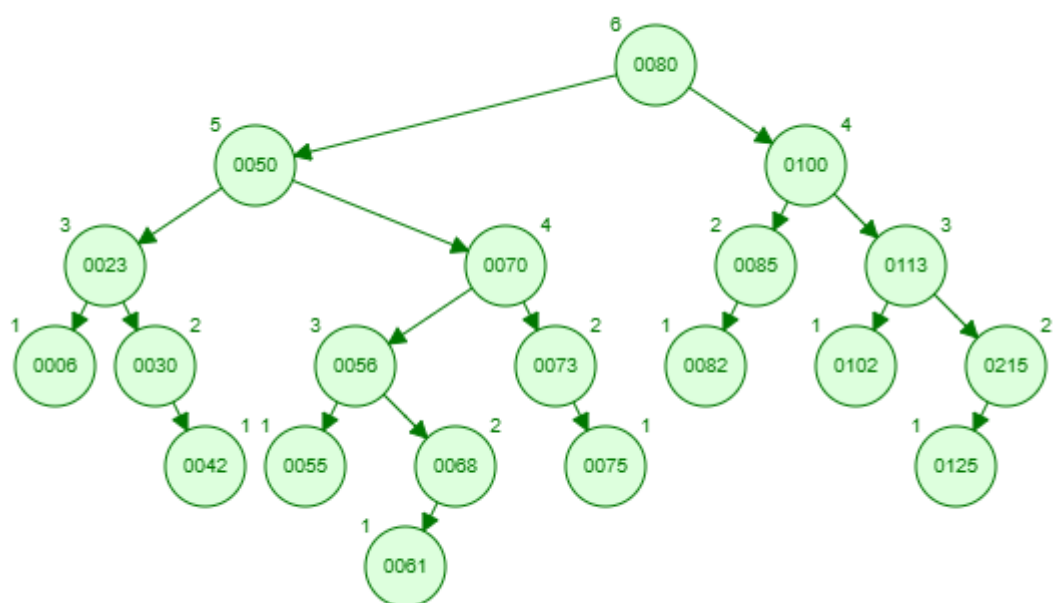
Вставим элемент 23:



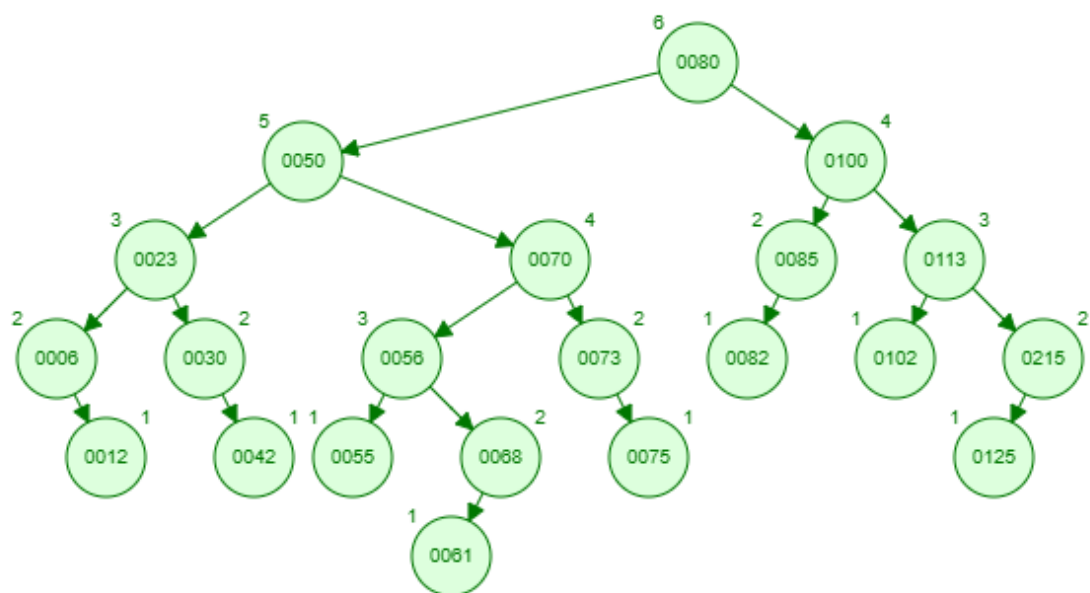
Вставим элемент 42:



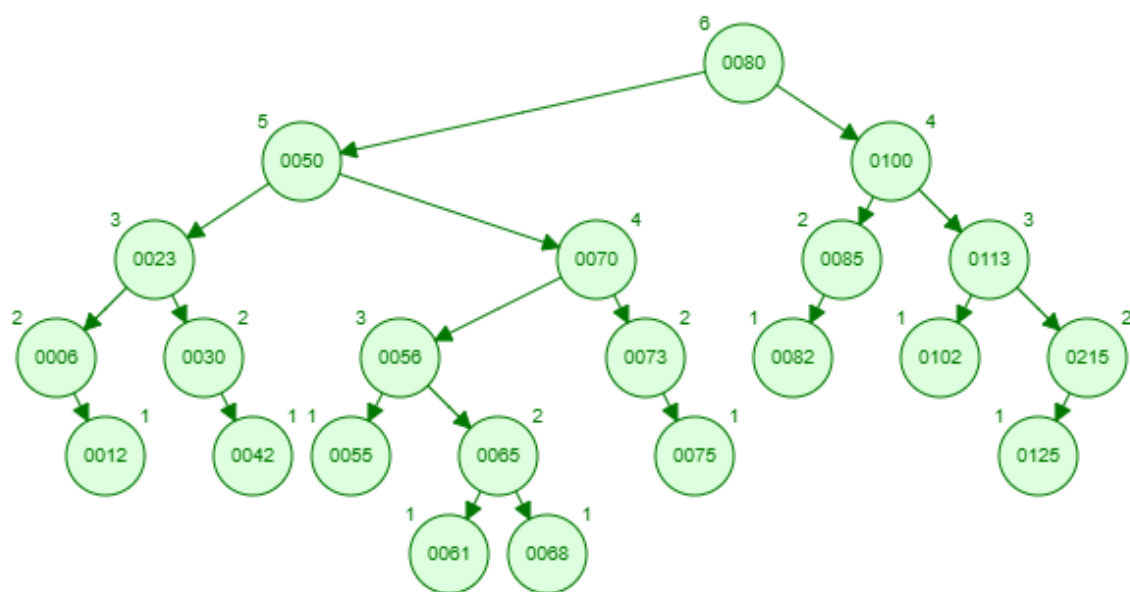
Вставим элемент 61:



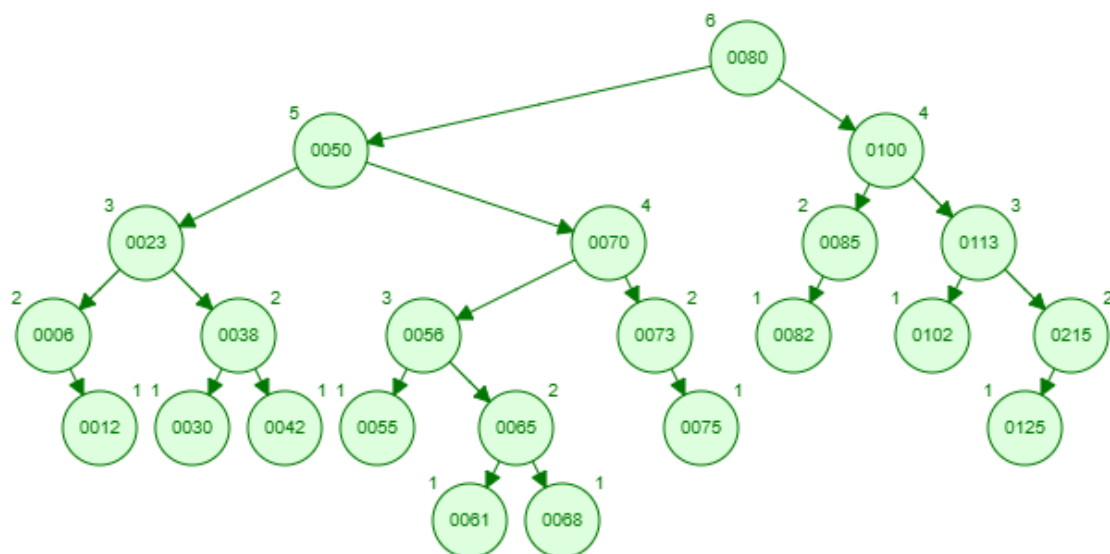
Вставим элемент 12:



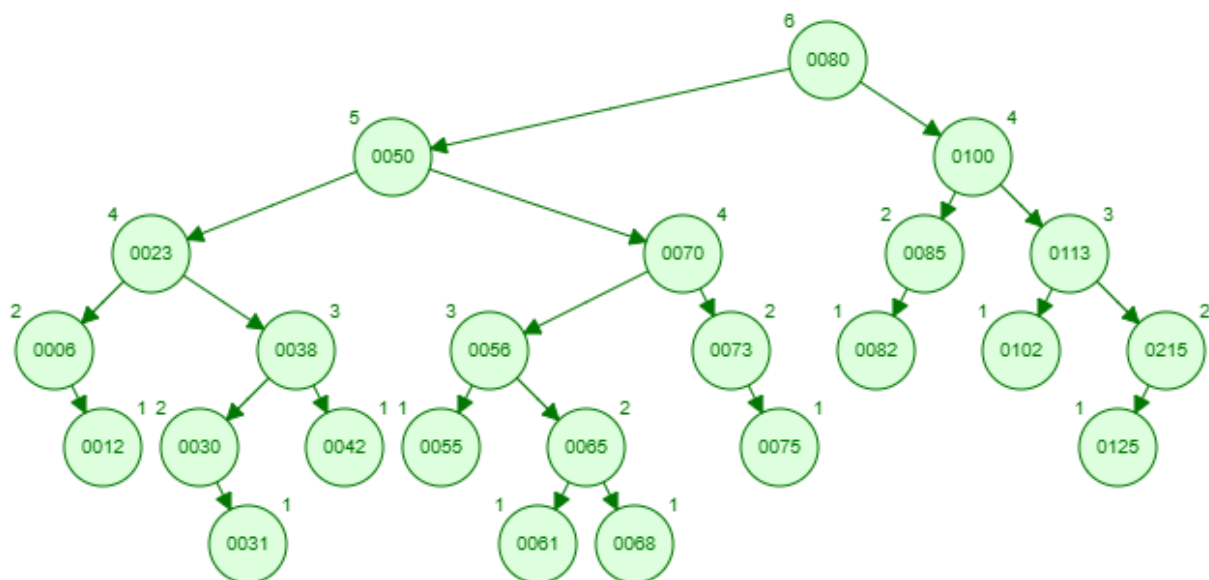
Вставим элемент 65:



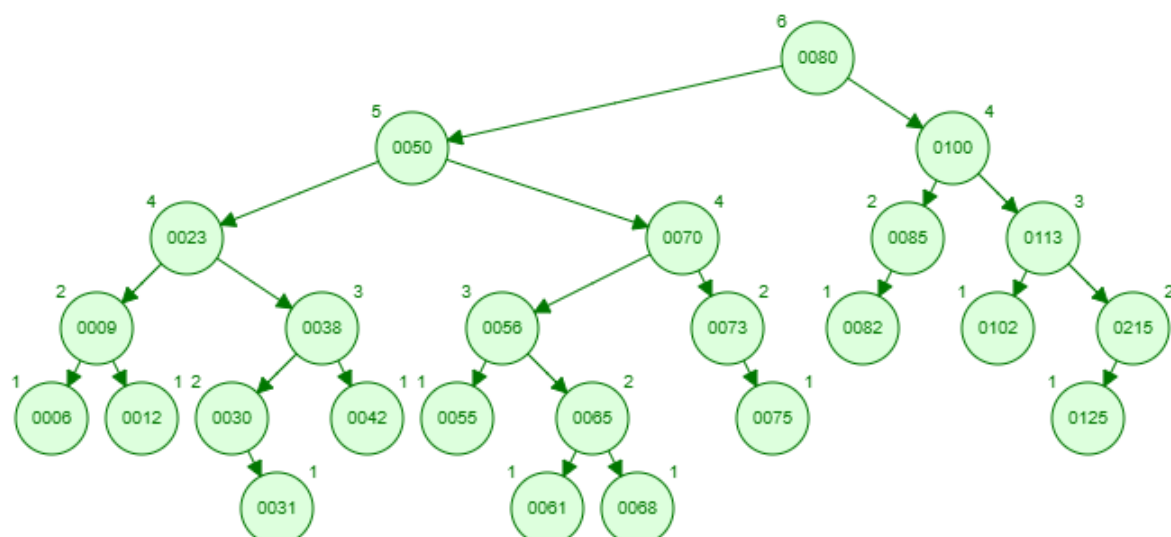
Вставим элемент 38 в левое поддереву первого уровня:



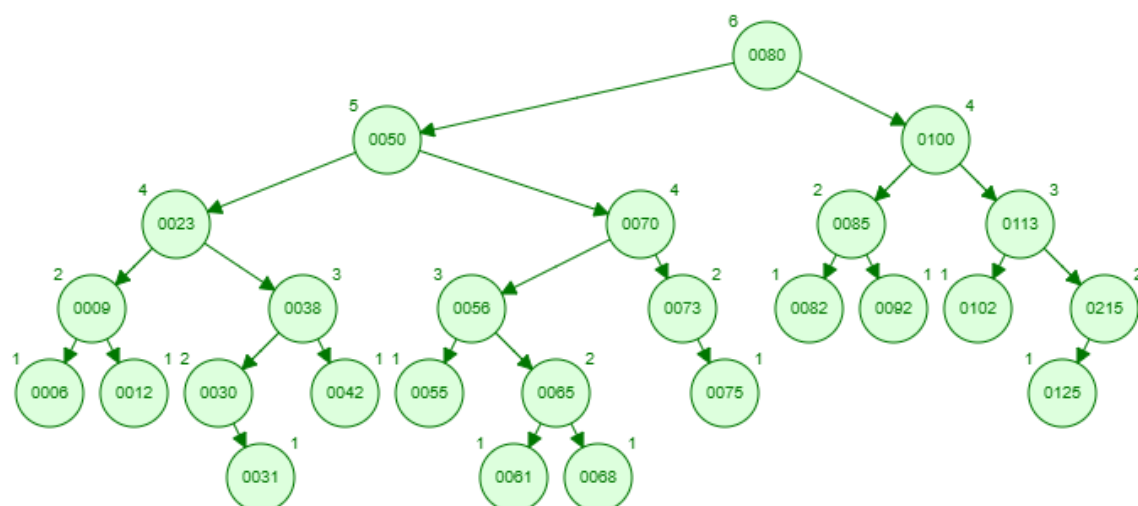
Вставим элемент 31 в левое поддереву первого уровня:



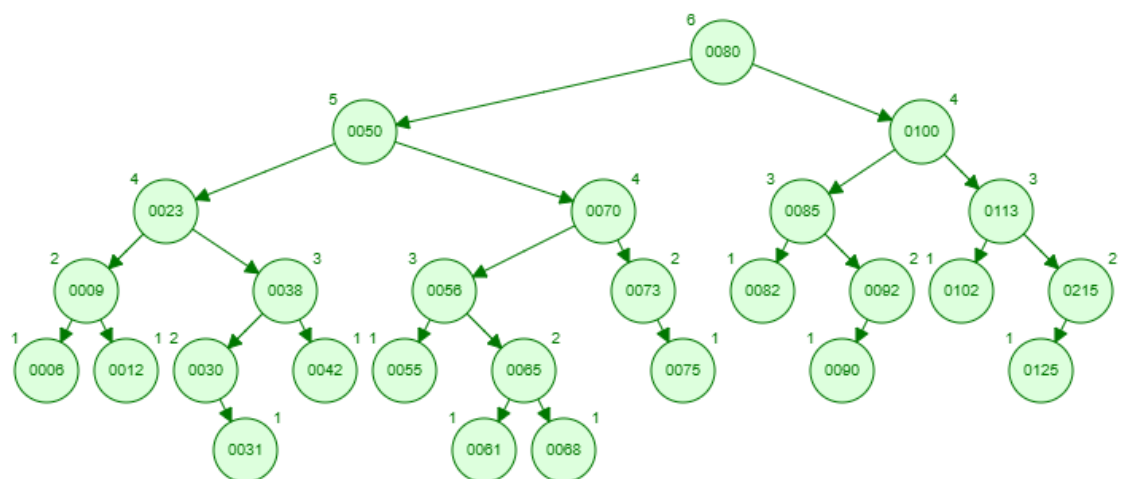
Вставим элемент 9:



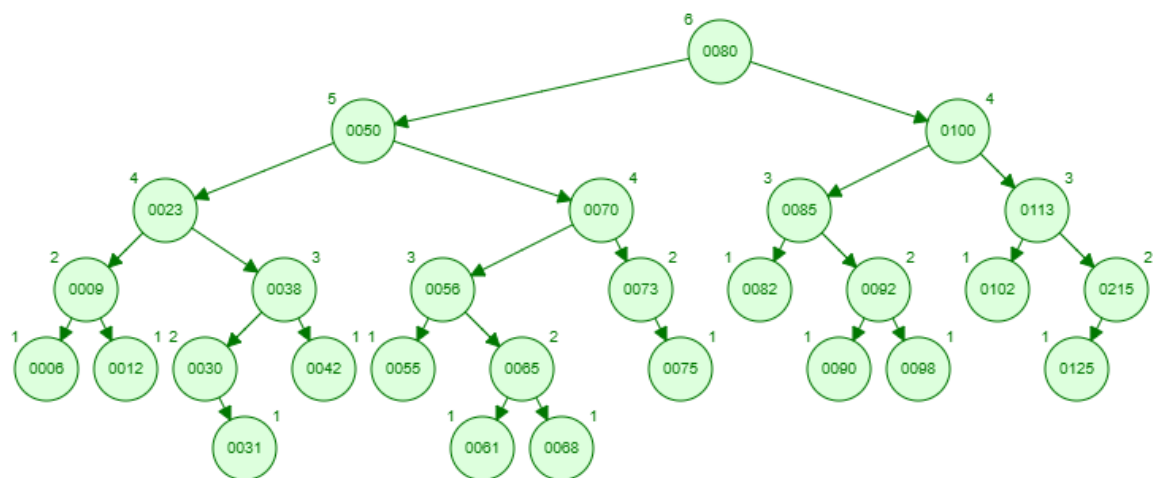
Вставим элемент 92:



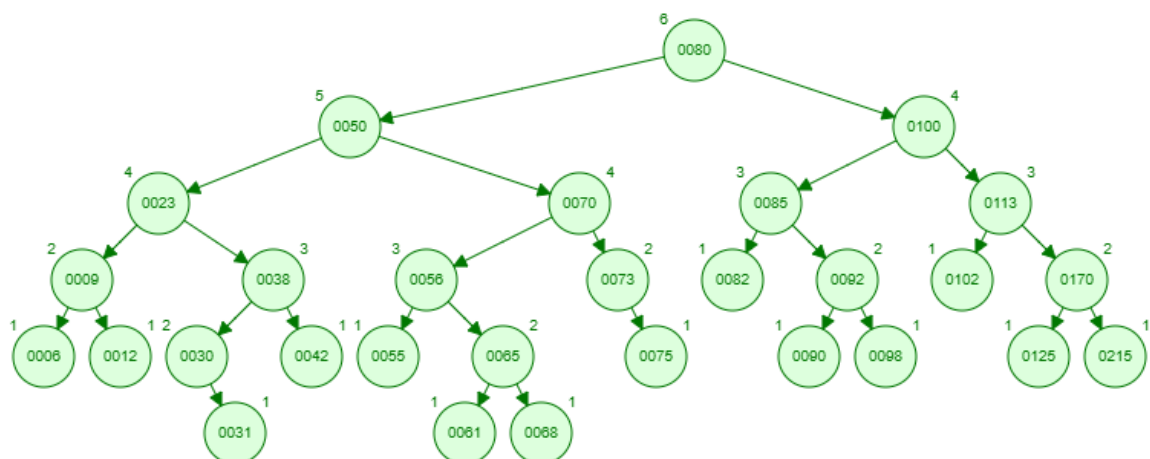
Вставим элемент 215:



Вставим элемент 23 в левое поддереву первого уровня:

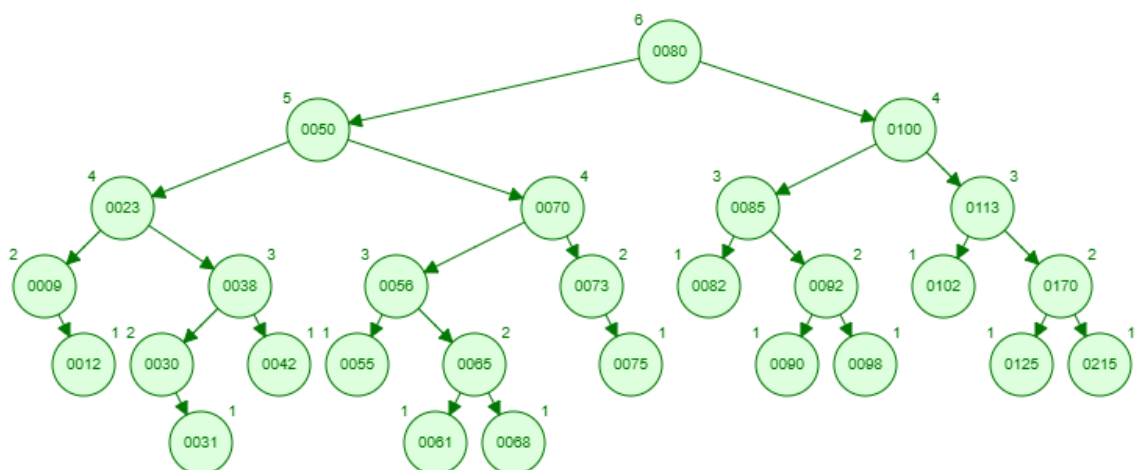


Вставим элемент 170:

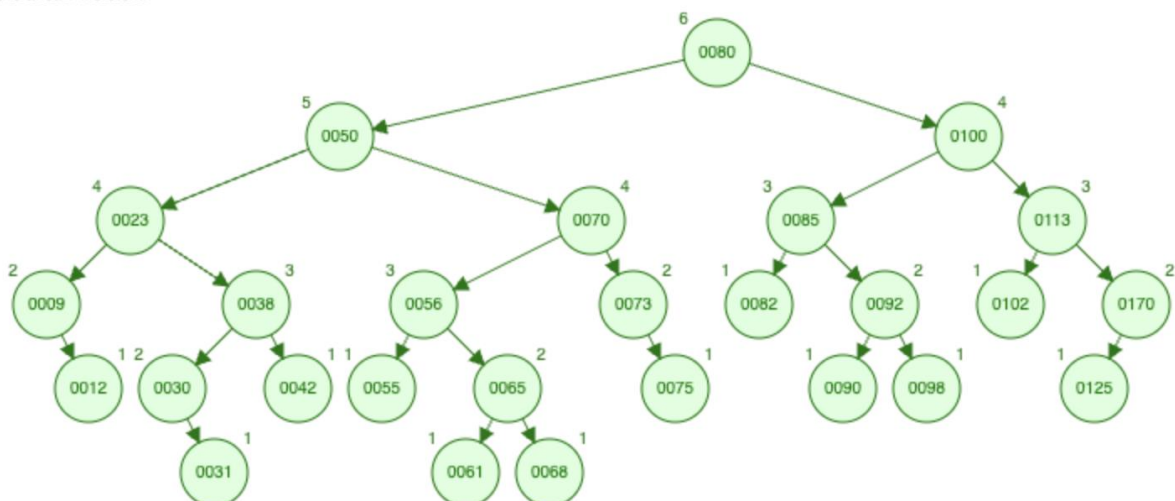


Удаление

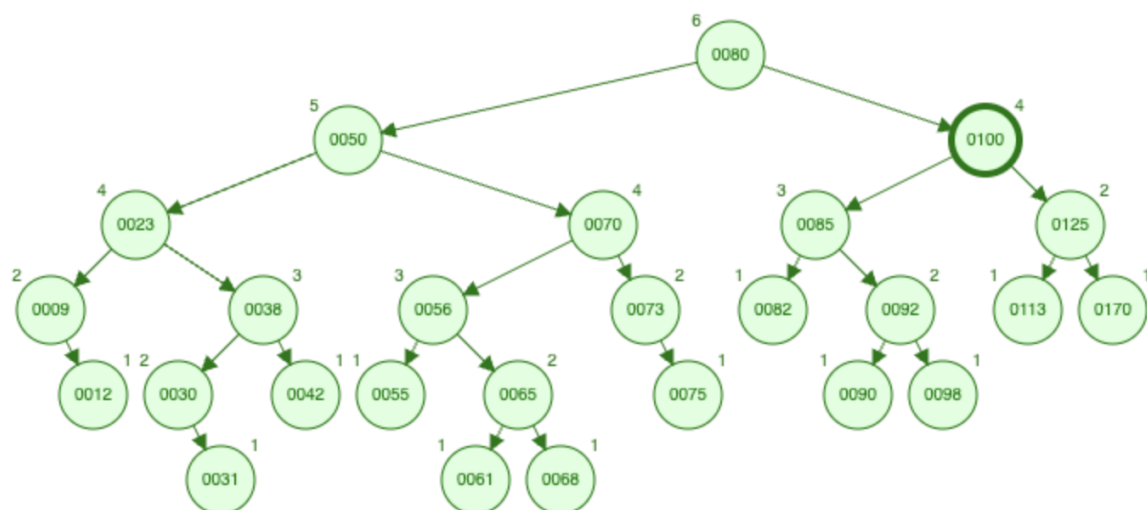
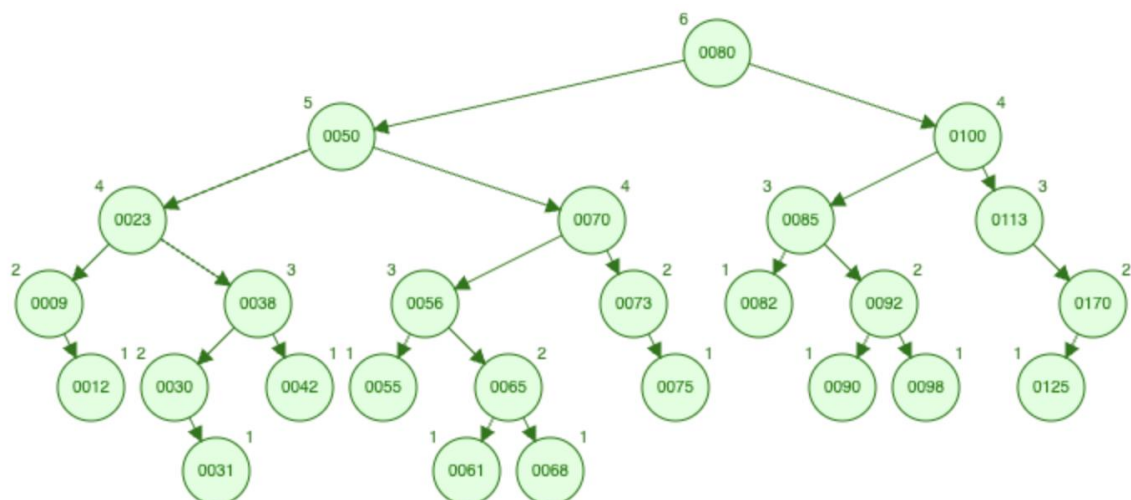
Для 6:



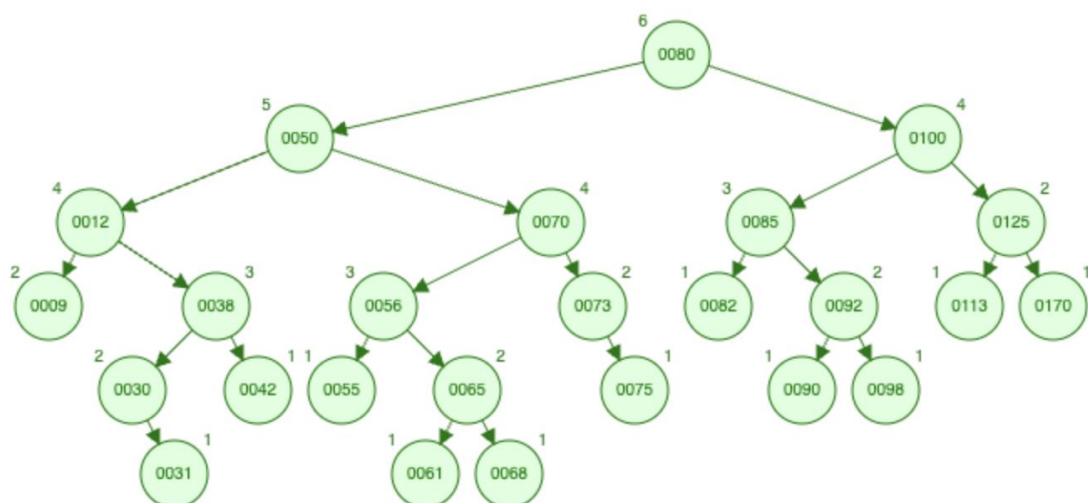
Для 215:

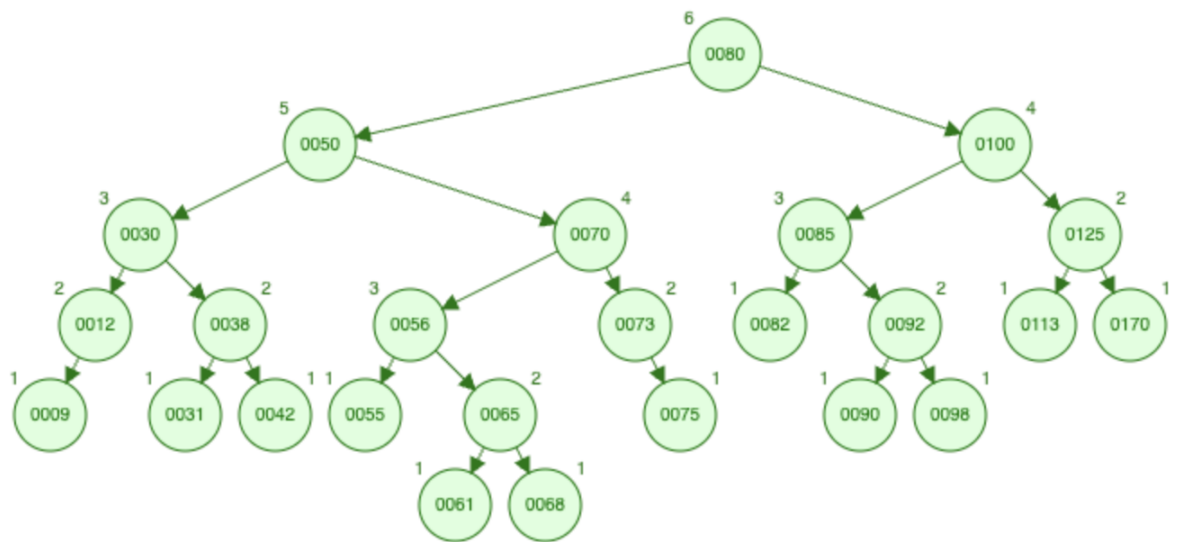


Для 102:



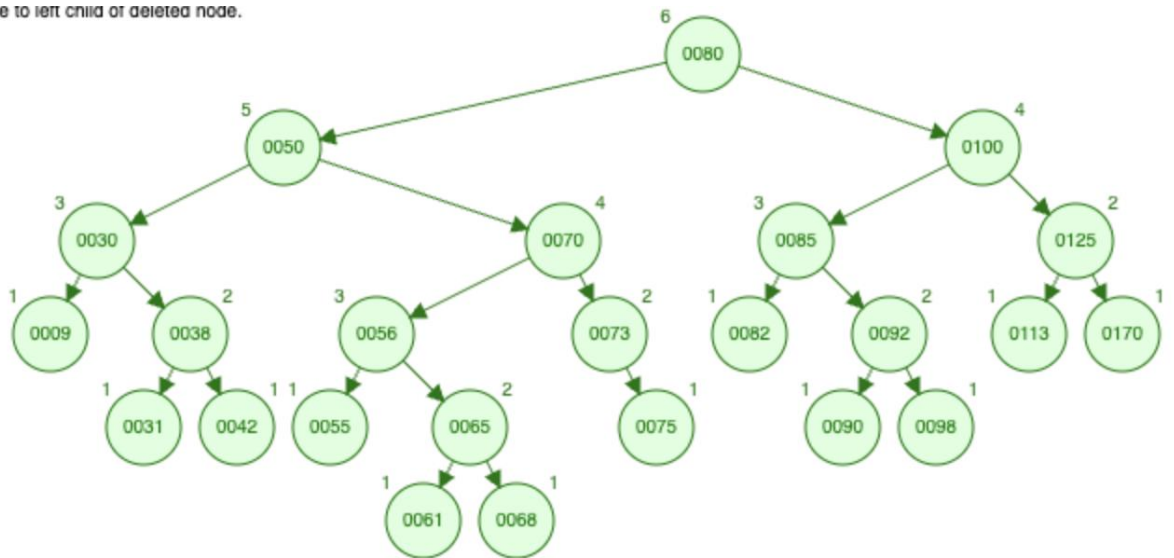
Для 23:



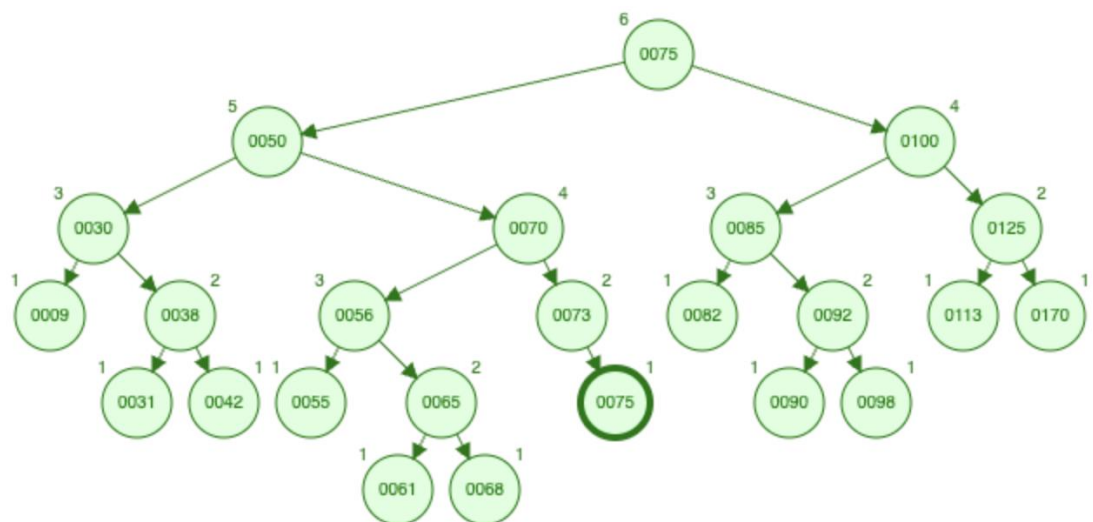


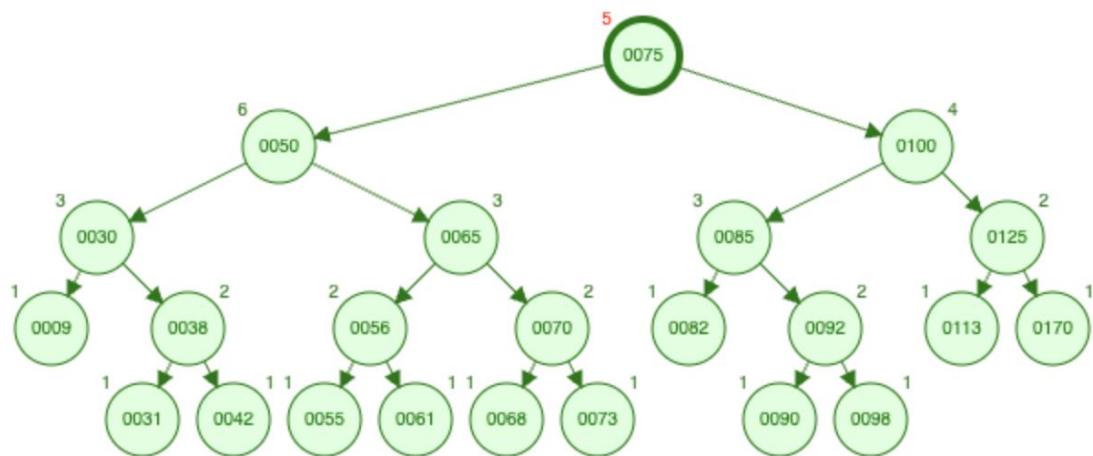
Для 12:

need node to left child of deleted node.



Для 80:





После удаления корня его место занимает либо максимальный элемент из левого поддерева, либо минимальный из правого. В моём случае новым корнем стал элемент 75 т.е. – максимальный из левого поддерева.

Построить В-дерево (порядка 2), включая элементы в указанной последовательности по столбцам.



Вставим элемент 55:



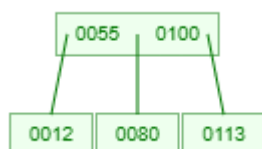
Вставим элемент 12, разобьем дерево на два поддерева:



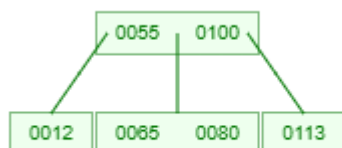
Вставим элемент 80:



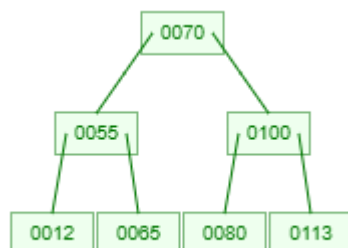
Вставим элемент 113, разобьем на два поддерева, средний элемент поставим в родительский узел:



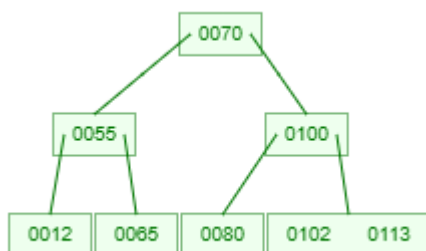
Вставим элемент 65:



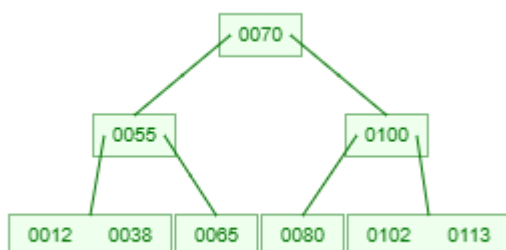
Вставим элемент 70, произойдет два разбиения подряд:



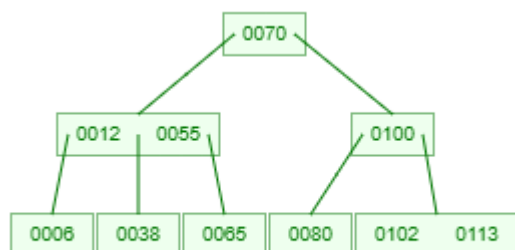
Вставим элемент 102:



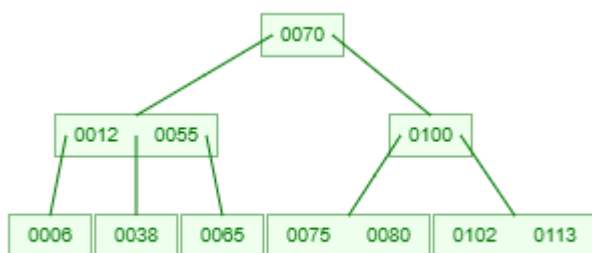
Вставим элемент 38:



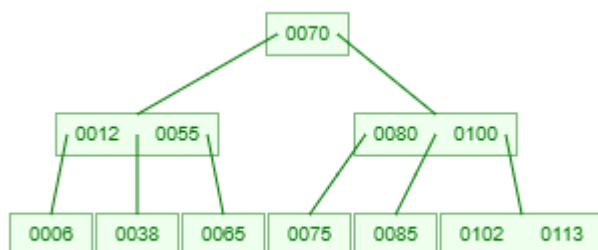
Вставим элемент 6, произойдет разбиение:



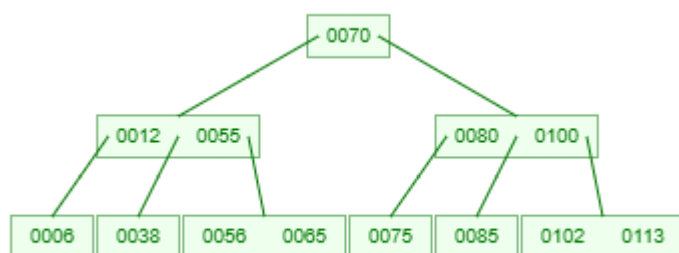
Вставим элемент 75:



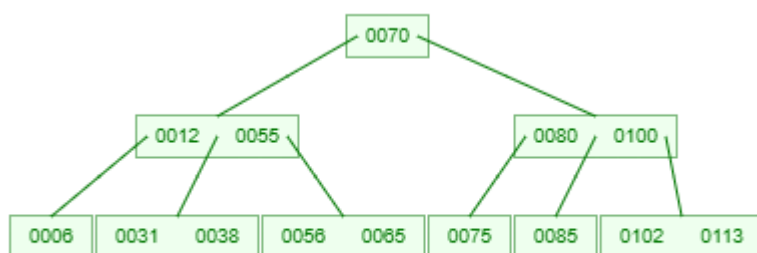
Вставим элемент 85:



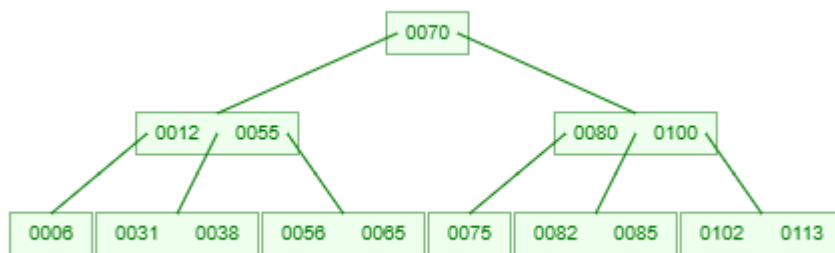
Вставим элемент 56:



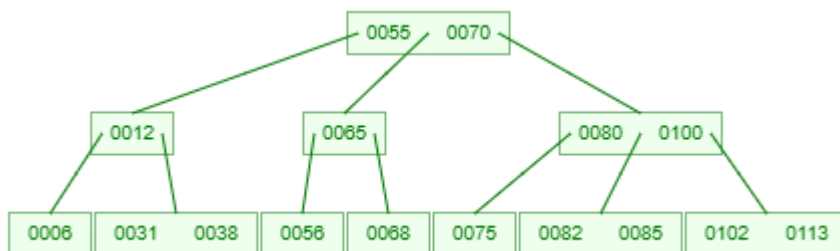
Вставим элемент 31:



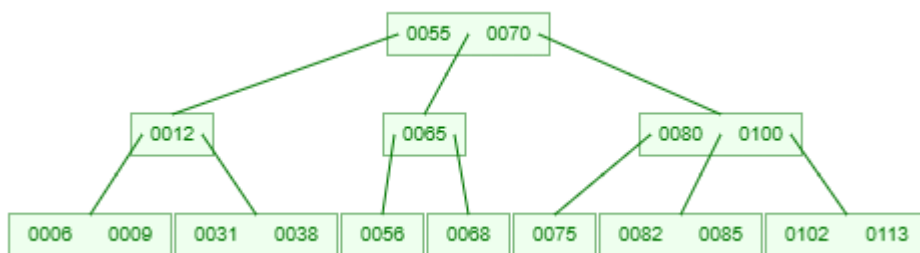
Вставим элемент 82:



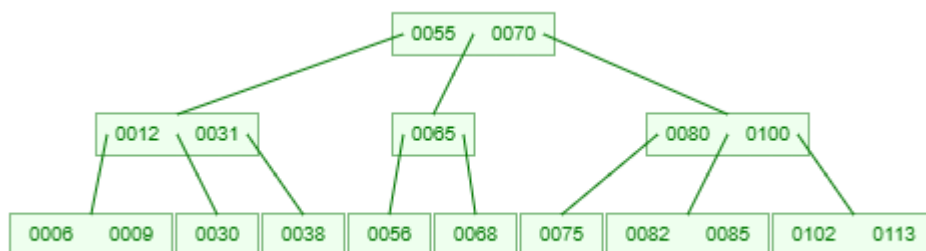
Вставим элемент 68:



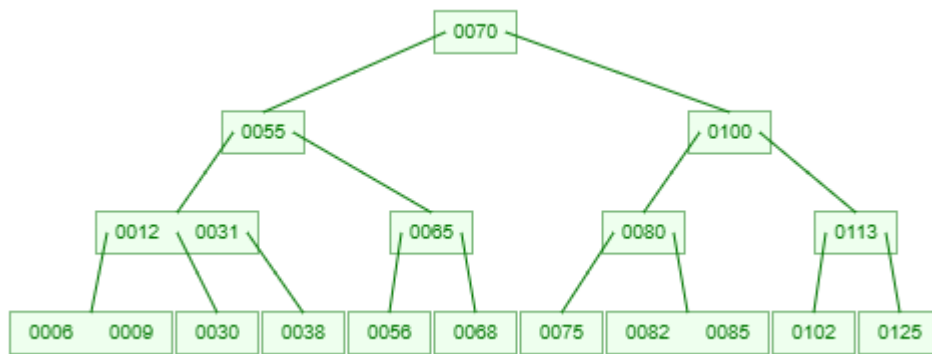
Вставим элемент 9:



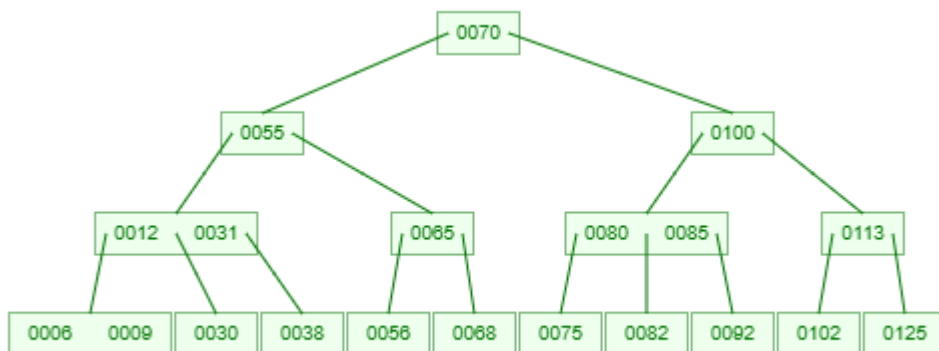
Вставим элемент 30, произойдет разбиение:



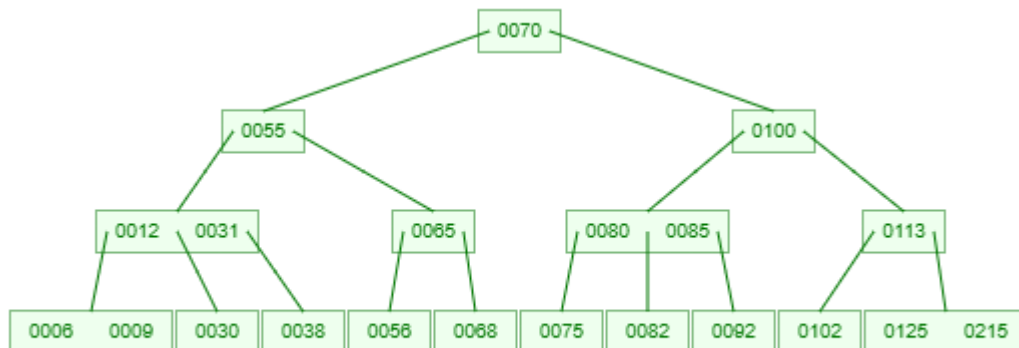
Вставим элемент 125:



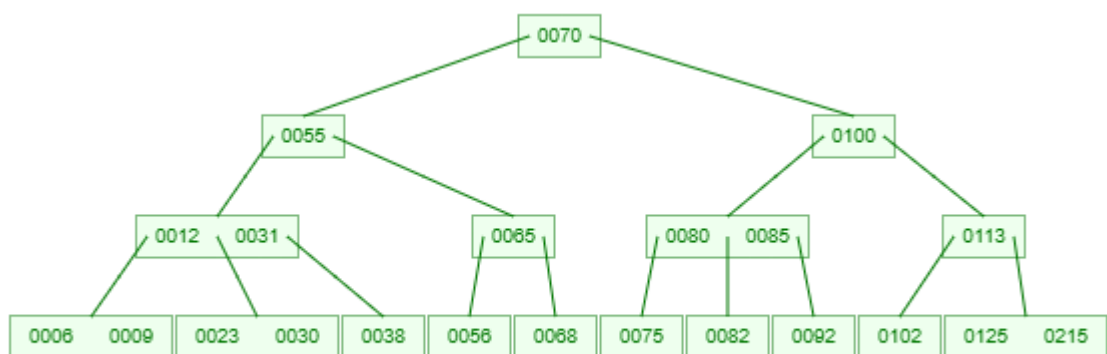
Вставим элемент 92:



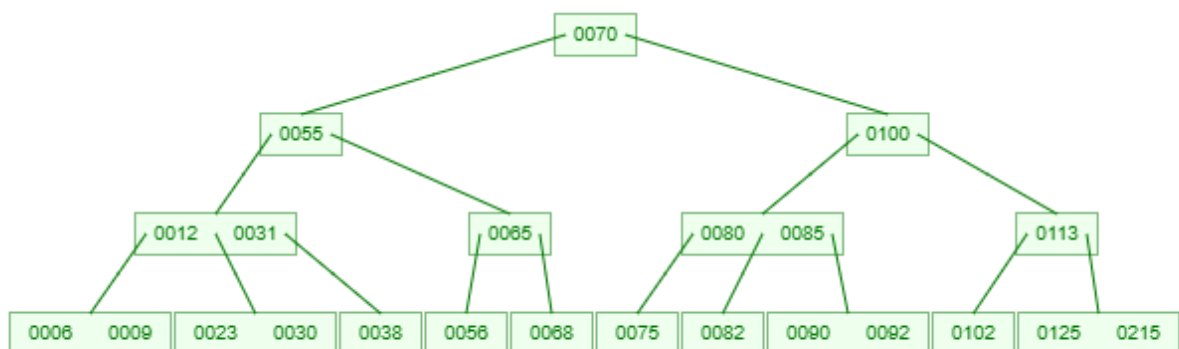
Вставим элемент 215, произойдет разбиение:



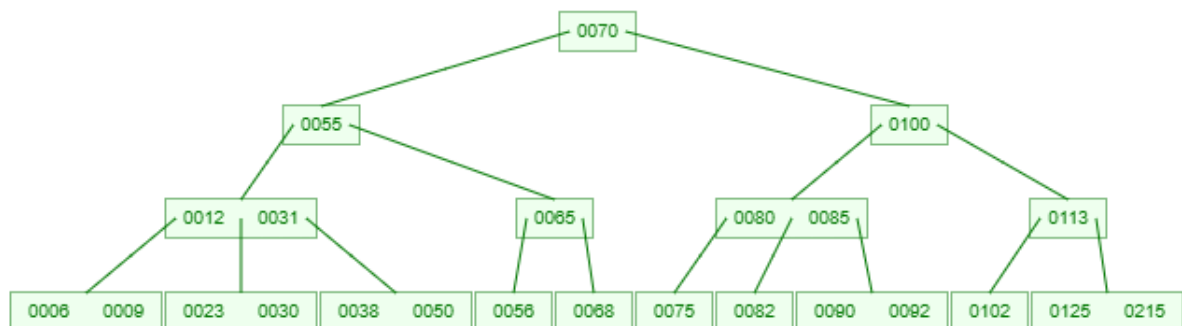
Вставим элемент 23:



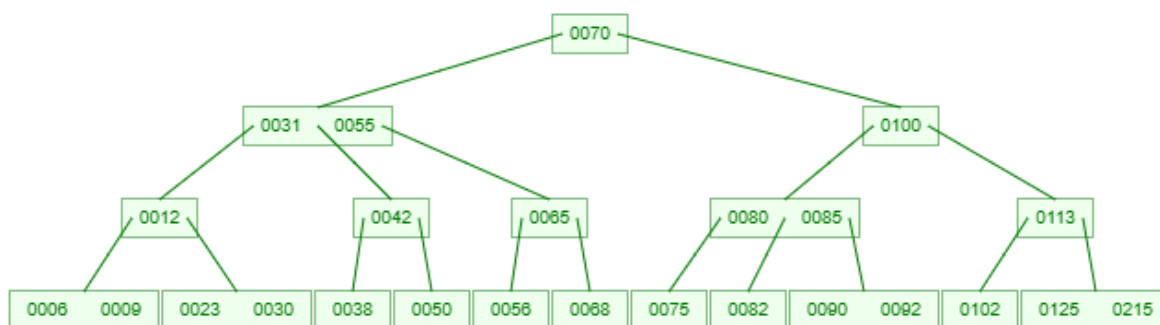
Вставим элемент 90, произойдет два разбиения:



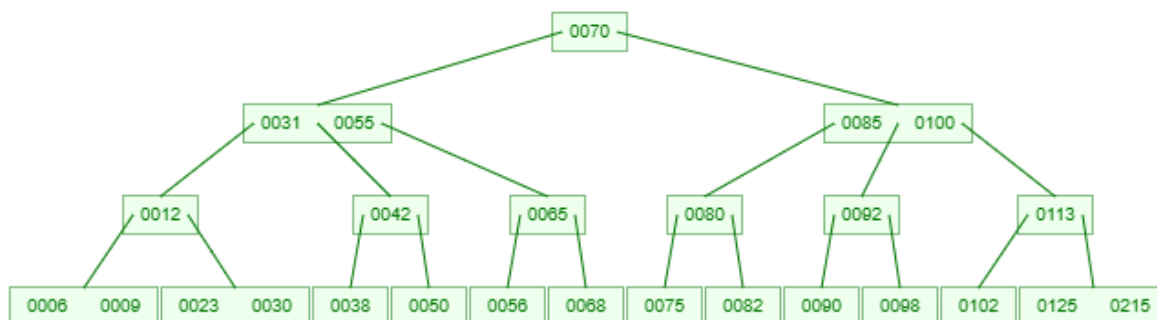
Вставим элемент 50:



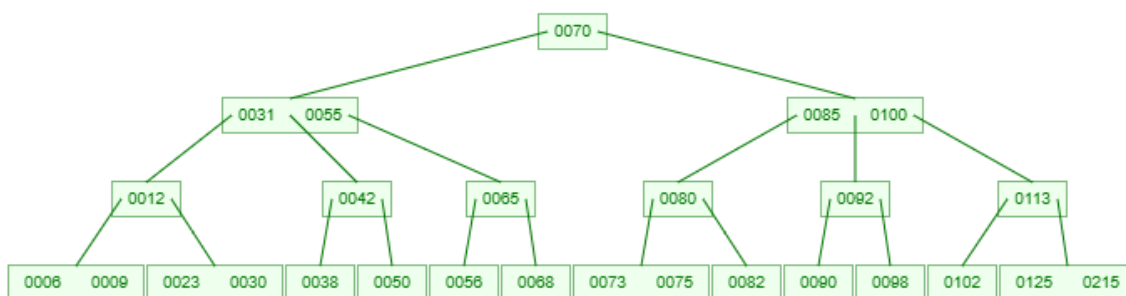
Вставим элемент 42, произойдет два разбиения:



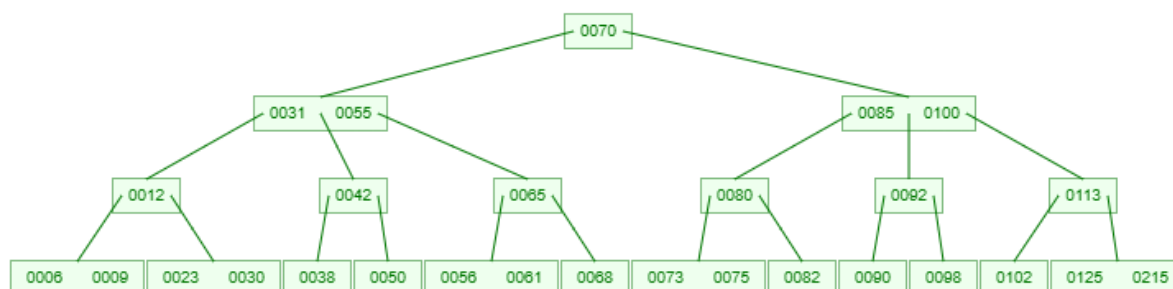
Вставим элемент 98:



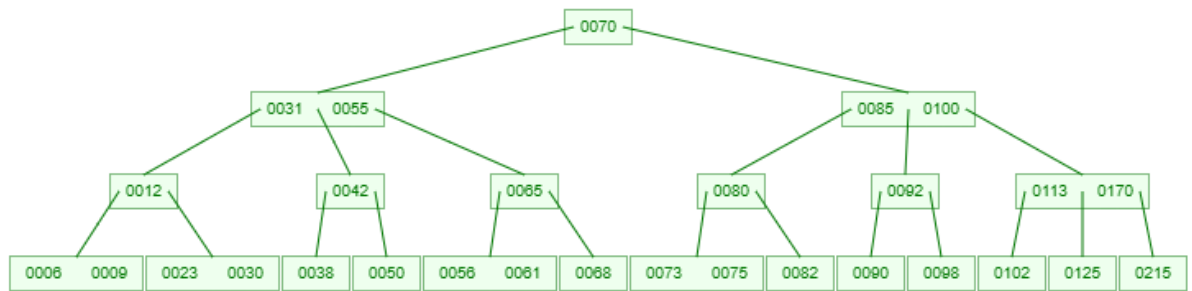
Вставим элемент 73:



Вставим элемент 61:

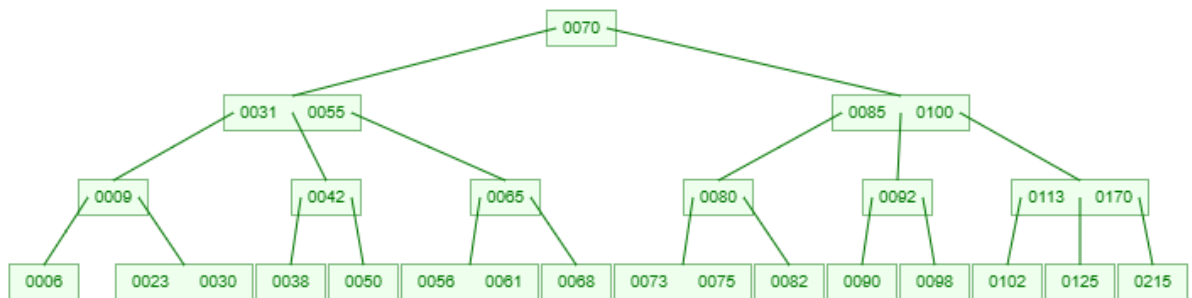


Вставим элемент 170:

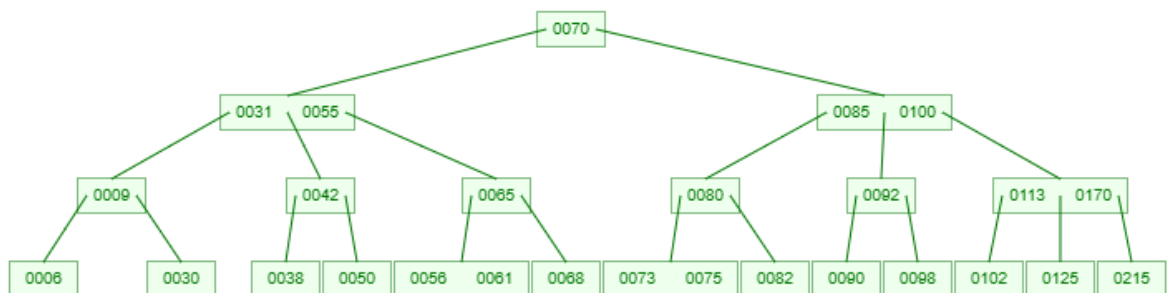


Удаление:

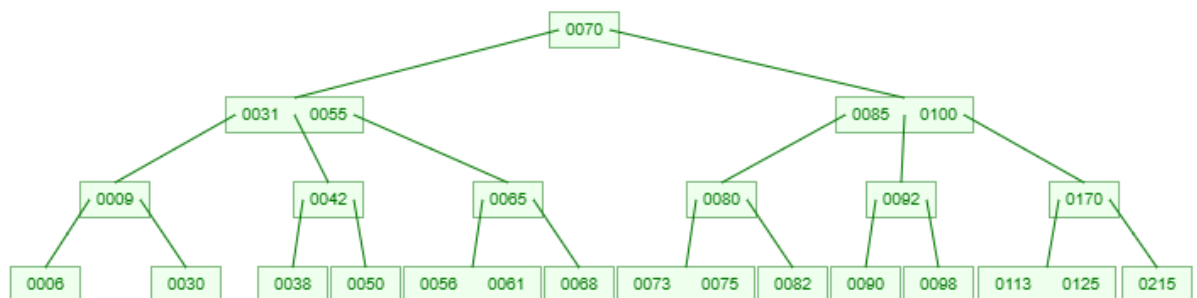
Удалим элемент 12:



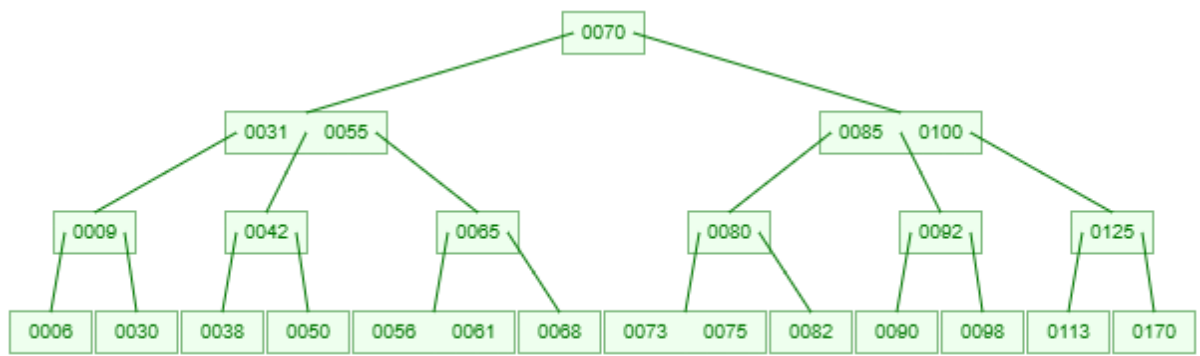
Удалим элемент 23:



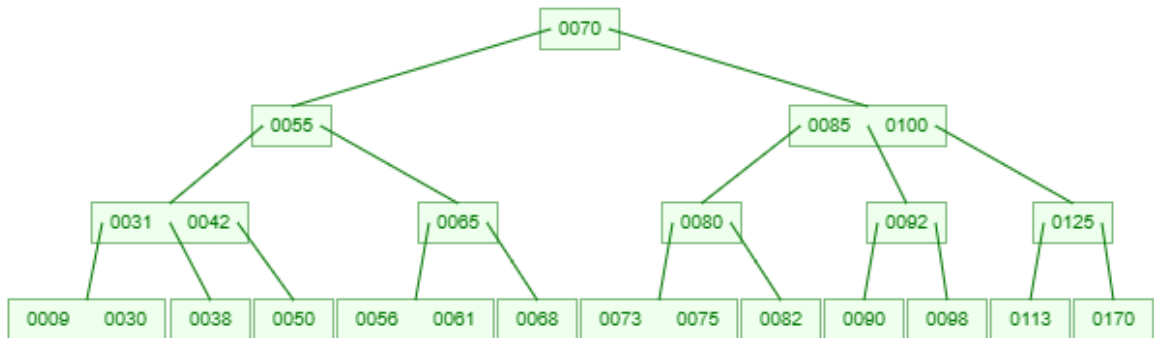
Удалим элемент 102:



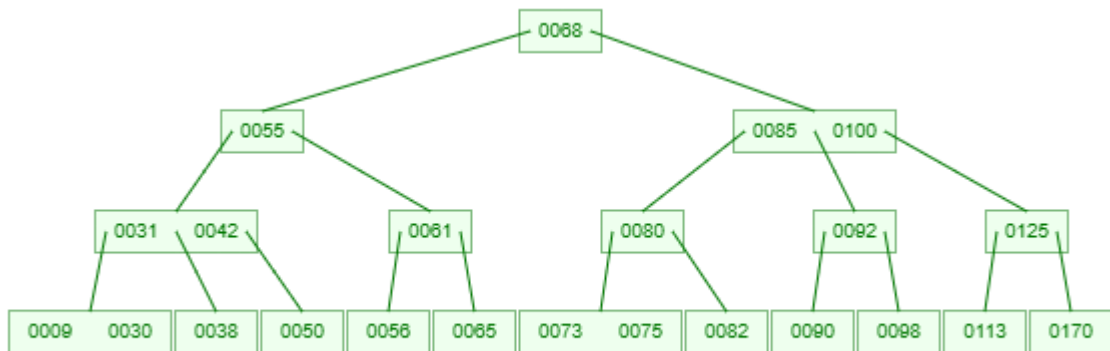
Удалим элемент 215:



Удалим элемент 6:



Удалим теперь корневую вершину:



5. Вывод

В рамках выполнения лабораторной работы я изучил механизмы функционирования динамических структур данных на примере AVL-деревьев и В-деревьев. Мне удалось реализовать программные алгоритмы для поэтапного наполнения структур, корректного удаления элементов и автоматического поддержания баланса. При работе с AVL-деревом я на практике убедился, что за счет использования механизма вращений (одинарных и двойных) дерево сохраняет строгую логарифмическую сложность $O(\log n)$ даже при интенсивной модификации данных. В то же время реализация В-дерева второго порядка показала другой подход к оптимизации: здесь стабильность структуры обеспечивается не вращениями, а разделением и слиянием узлов, при этом все листовые элементы всегда остаются на одной глубине. Сравнительный анализ этих моделей позволил глубже понять, как различные способы балансировки влияют на эффективность хранения и поиска информации в памяти.